Professores:

Celso Giusti

Daniel Manoel Filho

Isadora Dias Afonso

Marlon Palata Fanger Rodrigues

JSX e Estrutura Básica

SENAI

Capacidades Técnicas Abordadas

3. Interpretar os requisitos do sistema, tendo em vista a elaboração dos componentes em ambiente mobile

4. Definir os elementos de entrada, processamento e saída para a codificação das funcionalidades mobile

Introdução

Entendendo a estrutura

```
meu-projeto/
├── assets/          # Imagens, fontes e ícones
├── node_modules/    # Dependências do projeto (npm/yarn)
├── .gitignore       # Arquivos que o Git deve ignorar
├── App.js           # O componente principal da sua aplicação
├── app.json         # Configurações globais do Expo
├── index.js         # Ponto de entrada (Entry Point) do app
└── package.json     # Scripts e lista de bibliotecas instaladas
```

Introdução

O que é JSX?

JSX é uma extensão da sintaxe do Java Script que permite escrever código que **parece HTML**, mas na verdade é Java Script Puro.

Sem o JSX escrever o código da interface fica muito mais complexo.

SEM JSX

```
export default function Saudacao() {
  return React.createElement(
    View,
    null,
    React.createElement(Text, null, "Olá, SENAI!"),
  );
}
```

COM JSX

```
export default function Saudacao() {
  return (
    <View>
      <Text>Olá, SENAI!</Text>
    </View>
  );
```

Introdução

Regras do JSX

Para um bom funcionamento do **JSX** devemos seguir algumas regras na construção da tela

Vamos falar de 3 regras principais individualmente.

Regra 1

Retorno de um ÚNICO elemento

Todo componente deve retornar um único elemento que envolva **todo conteúdo**. Caso exista mais de um componente **PAI**.

Normalmente esse componente vai ser um **fragment (<> </>)** ou uma <**View** >

O **fragment** é um elemento invisível que agrupa outros elementos sem adicionar nada na tela.

```
export default function App() {
  return (
    <View style={styles.container}>
      <Text>Hello</Text>
    </View>
  );
```

Regra 2

Fechamento de Tags

Diferente do HTML que permite que algumas tags fiquem em aberto, no React Native isso não é possível, **TODAS** as tags devem ser fechadas.

Alguns componentes tem o que chamamos de Self-Closing

✗ `<Image source={img}>`	✓ `<Image source={img} />`
✗ `<TextInput>`	✓ `<TextInput />`
✗ `<View></View>  (sem filhos)`	✓ `<View />`

Regra 3

camelCase para atributos e estilos

No JSX, atributos e propriedades de estilo usam camelCase em vez de kebab-case:

Conceito	CSS / HTML	React Native
Cor de fundo	`background-color`	`backgroundColor`
Tamanho de fonte	`font-size`	`fontSize`
Borda arredondada	`border-radius`	`borderRadius`
Classe CSS	`class="btn"`	`style={styles.btn}`

Estilos são objetos JS — não existe arquivo CSS separado no React Native. Duplas chaves {{ }}: a primeira abre expressão JS, a segunda é o objeto de estilo.

Off Topic

Case Types

```
1  {
2      "camelCase": "camelCase",
3      "PascalCase": "PascalCase",
4      "kebab-case": "kebab-case",
5      "snake_case": "snake_case",
6      "SCREAMING_SNAKE_CASE": "SCREAMING_SNAKE_CASE",
7      "Train-Case-Name": "Train-Case-Name",
8      "UPPERCASE": "UPPERCASE"
9  }
```

Primeiro Componente

StyleSheet

O stylesheet é o responsável pela estilização de nossos componentes no React Native. Ele funciona de forma muito parecida com o CSS tradicional, porém é estruturado como objetos JavaScript.

Essa forma permite uma maior organização, separando a estilização da lógica de negócio, já que podemos nomear os objetos estilizados e aplicar nos componentes

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: "#fff",
    alignItems: "center",
    justifyContent: "center",
  },
});
```

Múltiplos Estilos

Podem existir casos onde você precise aplicar **mais de um** estilo em seu componente, para isso podemos utilizar o array de estilos.

O React Native permite que a prop style receba um array. O motor nativo processa esse array de forma linear (da esquerda para a direita).

```
<View style={[styles.base, styles.extra]} />
```

Como o React Native lê isso:

- Ele olha para o primeiro item;
- Ele olha para o segundo e, se houver conflito de propriedades (ex: ambos têm color), o último item do array sobrescreve o anterior.

JSX e Estrutura Básica

Expressões JS em JSX

O JSX só aceita expressões que retornam valor, e não statements (if, for, while)

Variáveis

`{nome}`

Operações

`{2026 - 22}`

Ternário

`{x ? 'A' : 'B'}`

Template String

`{`Olá, ${nome}`}`

O que NÃO pode dentro de { }

Estruturas de controle como if/else, for e while NÃO são expressões — não funcionam dentro das chaves. Use ternário ou lógica antes do return.

Renderização Condicional

O operador && permite renderizar algo SOMENTE se a condição for verdadeira. Se for falsa, nada aparece. Além disso é possível usar o ternário.

&& (mostrar ou não)

Usa quando precisa mostrar algo OU nada

```
{logado &&
  <Text>Bem-vindo!</Text>}
```

? : (escolher entre dois)

Usa quando precisa exibir A ou B

```
{logado
  ? 'Sair' : 'Entrar'}
```

Listas (.map)

No JSX não é possível utilizar laços de repetição como for e while dentro do return( ). Para iterar em listas na interface devemos usar o comando .map( ).

Nesse comando precisamos passar como parâmetro uma função de callback. Essa função será executada para cada item da lista, retornando um elemento JSX que o React converterá em interface.

Dentro da função de callback: Vamos receber o item e o index

```
const frutas = ["Maçã", "Banana", "Laranja"];
```

```
{frutas.map((fruta, indice) => (
          <Text key={indice}>
            {indice + 1}. {fruta}
          </Text>
))}
```

JSX e Estrutura Básica

Estrutura de Componentes Funcionais

Os componentes funcionais são essenciais para toda interface React Native, eles são funções JavaScript que retornam JSX.

1. **`import`**
   Importa os componentes nativos (View, Text...)

2. **`export const function`**
   Exporta a função para outros arquivos

3. **`Nome com MAIÚSCULA`**
   Obrigatório! MeuComponente, não meuComponente

4. **`return (...)`**
   Retorna o JSX que será renderizado

5. **`Lógica antes do return`**
   if/else, variáveis, cálculos — tudo ANTES do return

Exports e Imports

Temos aqui uma regra importante:

- Quando criamos um componente, é necessário realizar um **export** dele, para que outros arquivos no código tenham acesso a ele;
- Para utilizar ele em outro arquivo, é necessário então realizar um **import**.

```
1  import { StyleSheet, Text, TextInput, View } from "react-native";
2  import { BaseInput } from "../shared/components/BaseInput";
```

Atalhos VSCode

Como sabemos existem diversos atalhos que nos ajudam no desenvolvimento.

`Ctrl + Space`	Abre o autocomplete — mostra sugestões e já faz o import automático
`Ctrl + .`	Quick Fix — corrige erros de import automaticamente
`Alt + Shift + O`	Organiza imports (remove não usados, ordena)

REQUISITOS

Referências

React Docs. React Fundamentals

- Disponível em: https://reactnative.dev/docs/intro-react .

ESCOLA SENAI ÍTALO BOLOGNA

Av. Goiás, 139

Telefone

(11) 2396-1999

Instagram

@senai.itu

Facebook

/senaisp.itu

Site

https://sp.senai.br/unidade/itu/